Robust Retrieval-Augmented Generation via Textual-Gradient-Guided Adversarial Context Synthesis

Anonymous ACL submission

Abstract

Retrieval-Augmented Generation (RAG) systems often fail when retrieved contexts contain scattered evidence, distracting information, or severe noise, while existing offline robustness training methods struggle to capture diverse and query-specific retrieval failures in real-world scenarios. To address this limitation, we propose TAGS (Textual-Gradient-Guided Adversarial Context Synthesis), a framework for improving the robustness of Retrieval-Augmented Language Models (RALMs) through dynamic adversarial context synthesis. TAGS generates factually correct yet highly challenging adversarial contexts that preserve answer-supporting evidence while increasing the difficulty for RALMs to exploit such evidence. Through an iterative generation–evaluation–feedback process, TAGS converts model feedback into textual gradients to guide adversarial context optimization without relying on predefined noise construction rules. The optimized adversarial contexts are then used for robust fine-tuning, enabling RALMs to better handle noisy and difficult retrieval scenarios. Extensive experiments on multiple open-domain question answering benchmarks demonstrate that TAGS substantially improves the robustness and accuracy of RALMs under varying retrieval noise levels and out-of-domain settings. Code is available at https://anonymous.4open.science/r/TTT_ttlr.

1 Introduction

Large language models (LLMs) have achieved remarkable success in natural language understanding and generation tasks (**???**). However, in knowledge-intensive scenarios, LLMs still suffer from several limitations, including hallucination, outdated knowledge, and insufficient domain coverage (**??**). Retrieval-Augmented Generation (RAG) mitigates these issues by incorporating external documents as contextual knowledge and has become a widely adopted framework for open-domain question answering and knowledge-intensive applications (**???**).

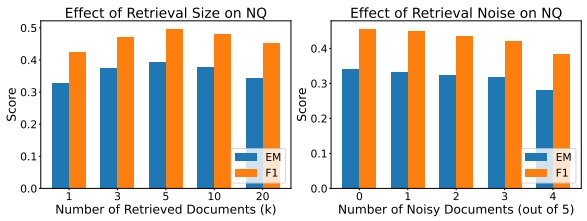


Figure 1: Sensitivity analysis on Natural Questions. Left: performance as retrieval depth k increases. Right: performance as answer-supporting documents are progressively replaced by low-ranked distractors.

Despite its effectiveness, the performance of RAG systems is highly dependent on the quality and usability of retrieved contexts. Previous studies have shown that retrieval relevance alone is insufficient to guarantee reliable generation: even when retrieved documents contain the correct answer, models may still fail to extract useful evidence when the context contains distracting information, noisy documents, or complex semantic interference (**??**). As retrieval noise increases, RAG models often experience significant performance degradation, indicating that retrieval defects affect not only relevance but also the stability of model reasoning (**??**).

To improve the robustness of RAG systems, existing studies mainly focus on offline training strategies, including noise-aware fine-tuning, adversarial training, and context-aware reasoning mechanisms (**????**). However, these approaches usually rely on manually designed perturbations or predefined noise distributions, which limits their ability to cover the diverse and evolving failure patterns encountered in real-world retrieval systems. In particular, retrieval failures are often query-specific and context-dependent, making it difficult for a fixed training distribution to expose all potential weaknesses of RAG models.

This motivates a fundamental question: *Can we automatically discover context-specific weaknesses of RAG models and synthesize targeted training data according to model feedback?*

Recent studies suggest that model behaviors and generation feedback provide valuable signals for improving text generation and optimization (**??**). Instead of exhaustively enumerating possible retrieval failures during data construction, leveraging feedback from model responses under challenging contexts provides a promising direction for constructing more effective robustness training data.

In this work, we propose TAGS (Textual-Gradient-Guided Adversarial Context Synthesis), a framework for improving the robustness of Retrieval-Augmented Language Models (RALMs). The key idea of TAGS is to dynamically synthesize challenging adversarial contexts based on model feedback and transform the discovered weaknesses into effective robustness supervision.

Specifically, TAGS introduces a frozen attacker model to generate candidate adversarial contexts conditioned on the query and the original retrieval context. These adversarial contexts preserve answer-supporting evidence while making such evidence more difficult for RALMs to reliably utilize. Through an iterative **generation–evaluation–feedback process**, TAGS evaluates candidate contexts according to the target RALM's behaviors, converts preference signals into natural-language **textual gradients**, and uses them to guide subsequent adversarial context optimization.

After iterative optimization, the generated adversarial contexts are filtered and incorporated into robust fine-tuning, enabling RALMs to better handle noisy and difficult retrieval scenarios. Unlike existing robustness methods that depend on predefined noise construction rules, TAGS dynamically identifies and synthesizes query-specific challenging contexts according to model feedback, providing more effective supervision for improving RAG robustness.

In this work, *adversarial contexts* refer to semantically modified retrieval contexts that preserve answer-supporting evidence while increasing the difficulty for the target RALM to exploit such evidence. They are designed to simulate challenging retrieval conditions, such as evidence dilution, information reordering, and distracting content, rather than malicious attacks such as corpus poisoning, malicious evidence injection, or targeted attacks on retrieval systems.

The main contributions of this work are summarized as follows:

- We propose TAGS (Textual-Gradient-Guided Adversarial Context Synthesis), a framework for RAG robustness enhancement that dynamically constructs query-specific adversarial contexts through an iterative generation–evaluation–feedback process.
- We introduce textual gradients as optimization signals for adversarial context synthesis, enabling the generation of factually correct yet highly challenging contexts without relying on predefined noise construction rules.
- Extensive experiments on multiple open-domain question answering benchmarks demonstrate that TAGS substantially improves the robustness and accuracy of RALMs under varying retrieval noise levels and out-of-domain settings.

2 Related Work

2.1 Robustness of Retrieval-Augmented Generation

Irrelevant or difficult-to-use retrieval contexts can substantially degrade answer generation, and semantically related but misleading contexts can be particularly harmful (**???**). RetRobust improves robustness by training with both relevant and irrelevant contexts to mitigate the impact of distracting retrieval results (**?**). Other approaches leverage note-based relevance reasoning (**?**) or instruction and rationale tuning (**?**). RbFT enhances robustness through defect-aware training with sampled or synthetically corrupted contexts (**?**). RAAT adopts adaptive adversarial training with noise recognition (**?**), while ATM improves robustness through iterative optimization between an attacker model and an answer generator (**?**). Different from these approaches that mainly rely on predefined perturbations or adversarial training objectives, TAGS dynamically synthesizes query-specific adversarial contexts through textual gradients and uses the synthesized contexts for robust fine-tuning of the target RALM.

2.2 Textual Feedback and Adversarial Context Synthesis

Recent studies demonstrate that natural-language feedback can serve as an effective optimization signal for text generation and refinement. TextGrad

introduces textual gradients to optimize text-valued variables through language-model-generated feedback (**?**). Test-Time Preference Optimization further explores iterative textual feedback for improving model responses without conventional parameter updates (**?**).

Different from previous methods that optimize generated responses or model behaviors, TAGS applies textual feedback to a new optimization target: adversarial contexts for robustness enhancement. Specifically, the target RALM's responses provide evaluation signals for ranking candidate contexts, while contrastive feedback between candidates guides subsequent adversarial context optimization. Parameter learning occurs only after context synthesis, through supervised fine-tuning on the selected adversarial contexts.

3 Method

We propose TAGS (Textual-Gradient-Guided Adversarial Context Synthesis), a framework for improving the robustness of Retrieval-Augmented Language Models (RALMs). The key idea of TAGS is to dynamically synthesize query-specific adversarial contexts based on model feedback and transform the discovered weaknesses into effective robustness supervision.

TAGS consists of two stages: (1) adversarial context synthesis, where a frozen auxiliary attacker model generates and optimizes challenging retrieval contexts through textual gradients; (2) robust fine-tuning, where the target RALM is optimized using the synthesized contexts.

During adversarial context synthesis, all model parameters remain frozen. Parameter updates are only performed in the subsequent supervised fine-tuning stage.

3.1 Problem Definition

Given a query q, a retrieved context $\mathcal{R}_K(q)$, and a reference answer y^*, TAGS aims to construct adversarial contexts that improve the robustness of RALMs under noisy and difficult retrieval scenarios.

An adversarial context d is defined as a semantically modified version of the original retrieval context $\mathcal{R}_K(q)$ that preserves answer-supporting evidence while increasing the difficulty for the target RALM to effectively utilize such evidence.

Unlike malicious retrieval attacks, TAGS focuses on realistic retrieval failures, including evidence dilution, information reordering, and distracting contexts. Therefore, adversarial contexts are designed to challenge evidence utilization rather than remove or inject unsupported knowledge.

3.2 Textual-Gradient-Guided Adversarial Context Synthesis

Given a training instance $(q, \mathcal{R}_K(q), y^*)$, TAGS first employs a frozen auxiliary attacker model A_ϕ to generate candidate adversarial contexts.

Initial context generation. The attacker receives the query, reference answer, and the complete retrieved context, and generates an initial candidate set:

$$\mathcal{D}_q^{(0)} = \{d_1^{(0)}, \ldots, d_n^{(0)}\}, \tag{1}$$

where

$$d_i^{(0)} \sim A_\phi(\cdot|q, \mathcal{R}_K(q), y^*). \tag{2}$$

Each candidate d_i contains a semantic rewrite of the complete top-K retrieved context. The attacker is instructed to preserve factual correctness and answer-supporting evidence while introducing challenges such as paraphrasing, information reordering, and evidence dilution.

Context evaluation and reward modeling. For each candidate adversarial context $d_i^{(t)}$, the frozen target RALM generates:

$$y_i^{(t)} = f_\theta(q, d_i^{(t)}). \tag{3}$$

The original retrieval context $\mathcal{R}_K(q)$ is not provided together with $d_i^{(t)}$, preventing the target RALM from bypassing the synthesized context by directly accessing the original evidence.

We evaluate each candidate using a weighted reward:

$$\begin{aligned} r(d_i, y_i, y^*) =& \alpha r_{\text{ans}}(y_i, y^*) \\ &+ \beta r_{\text{unc}}(y_i; q, d_i) \\ &+ \gamma r_{\text{evid}}(q, d_i, y^*). \end{aligned} \tag{4}$$

For simplicity, we denote the reward of a candidate context d_i as $r(d_i)$ hereafter. The three components are defined as:

$$r_{\text{ans}}(y_i, y^*) = -\left(EM(y_i, y^*) + F1(y_i, y^*)\right), \tag{5}$$

which encourages contexts that cause the target RALM to deviate from the reference answer.

$$r_{\text{unc}}(y_i; q, d_i) = \log(1 + PPL_\theta(y_i|q, d_i)), \quad (6)$$

which measures generation uncertainty under the candidate context.

$$r_{\text{evid}}(q, d_i, y^*) = \mathbb{I}[J(q, d_i, y^*) = \text{YES}], \quad (7)$$

which ensures that the candidate context still contains sufficient evidence supporting the reference answer.

The final reward balances three objectives: inducing generation difficulty, increasing model uncertainty, and preserving answer-supporting evidence. We use fixed weights across datasets: $\alpha = 1.0$, $\beta = 0.5$, and $\gamma = 2.0$.

Contrastive textual gradient optimization. At iteration t, TAGS maintains a candidate cache $\mathcal{C}_q^{(t)}$ containing previously evaluated contexts.

The highest- and lowest-reward contexts are selected:

$$d_t^+ = \arg\max_{d \in \mathcal{C}_q^{(t)}} r(d), \quad (8)$$

$$d_t^- = \arg\min_{d \in \mathcal{C}_q^{(t)}} r(d). \quad (9)$$

A frozen feedback model compares the two contexts and generates a textual gradient:

$$g_{\text{text}}^{(t)} = G(q, \mathcal{R}_K(q), d_t^+, d_t^-), \quad (10)$$

where the textual gradient describes why d_t^+ is more challenging than d_t^- while maintaining factual correctness. Unlike numerical gradients that update model parameters, textual gradients provide natural-language optimization directions for refining adversarial contexts.

Conditioned on the current best context and textual gradient, the attacker generates new candidates:

$$\mathcal{D}_q^{(t+1)} = \{d_j^{(t+1)}\}_{j=1}^N \sim A_\phi(\cdot|d_t^+, g_{\text{text}}^{(t)}). \quad (11)$$

This iterative generation–evaluation–feedback process continues until the synthesized contexts converge.

3.3 Robust Fine-tuning

After adversarial context synthesis, TAGS selects the highest-quality context:

$$d^* = \arg\max_{d \in \mathcal{V}_q} r(d), \quad (12)$$

where $\mathcal{V}_q$ denotes candidates passing the evidence-preservation check.

The selected triples:

$$(q, d^*, y^*) \quad (13)$$

form the supervised fine-tuning dataset.

We optimize the target RALM using LoRA with the standard supervised objective:

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(q,d^*,y^*)}[\log p_\theta(y^*|q, d^*)]. \quad (14)$$

During this stage, only the target RALM parameters are updated. The auxiliary attacker model and feedback model remain frozen.

3.4 Train–Test Separation

All adversarial context synthesis, reward computation, textual gradient generation, and LoRA updates are performed exclusively on training instances.

Held-out evaluation queries are never involved in context synthesis or model optimization.

During inference, TAGS only requires the retriever and the fine-tuned target RALM. No attacker model, textual gradient generation, reference answer, or additional parameter update is involved.

4 Experimental Settings

4.1 Tasks, Datasets, and RAG Setup

We evaluate on Natural Questions (NQ) (**?**), PopQA (**?**), HotpotQA (**?**), 2WikiMultiHopQA (**?**), and the DAE difficult-context evaluation set. DAE is held out from synthesis, candidate selection, and model updates. All systems use a common retrieve-then-read pipeline implemented with FlashRAG (**?**), the `e5-base-v2` dense retriever, and top-K retrieval with $K = 5$. We report exact match (EM) and token-level F1.

Following prior work, we apply one common context-sufficiency filter and evaluate every method on exactly the same retained instances (**?**). The filter removes examples for which the retrieved context does not support the reference answer, allowing the analysis to isolate whether the generator

can use available evidence rather than whether retrieval found it. No retained evaluation instance is used for synthesis or fine-tuning. Table **??** reports the resulting sizes.

4.2 Training and Evaluation Protocol

Adversarial context synthesis uses 3,000 labeled instances sampled exclusively from the *training* splits of NQ and HotpotQA. Synthesis prompts, reward computation, textual feedback, candidate selection, and LoRA updates are confined to those instances. Validation and test examples are excluded from synthesis and fine-tuning. The reward weights and optimization settings are fixed across datasets and are not selected using test performance. All reported held-out answers are generated after training, without further parameter updates.

4.3 Retrieval Noise Construction and Robustness Evaluation Settings

For each query, we retrieve the top-50 documents and keep the number of input documents fixed at five. We then progressively replace high-ranked documents with documents from ranks 47–50, producing the five settings in Table **??**. These low-ranked documents remain semantically close to the query under the same retriever but are less likely to contain answer-supporting evidence, so they serve as naturally retrieved distractors rather than synthetic malicious content. We apply this controlled evaluation to NQ, PopQA, HotpotQA, and 2WikiMultiHopQA.

4.4 Baselines

We compare against representative instruction-tuned LLaMA and Qwen models (**??**) and three robustness-oriented RAG methods: RbFT (**?**), ATM (**?**), and RAAT (**?**). All rows share the retriever, retrieved inputs, answer prompt, decoding procedure, context-sufficiency filter, and metrics.

Dataset	Test	Retain
Natural Questions (NQ)	3,610	982
PopQA	1,399	751
HotpotQA	7,404	348
2WikiMultiHopQA	12,576	210
DAE	982	774

Table 1: Statistics of the evaluated datasets after context sufficiency filtering. "Test" denotes the original test set size, and "Retain" denotes the number of instances retained for evaluation.

The robustness-training methods use LLaMA-3.2-3B-Instruct as the target backbone; the standalone LLaMA and Qwen rows are backbone controls and therefore retain their stated model sizes.

For trainable baselines, we follow the number of training examples and LoRA recipe reported by the corresponding original paper, including rank, scaling factor, learning rate, batch size, epochs, maximum sequence length, optimizer, and scheduler. This protocol compares complete published methods under a controlled test pipeline; it does not claim equal offline training compute. Appendix **??** provides further configuration details.

5 Results and Analysis

We organize the analysis around five research questions:

- **RQ1:** How does the proposed training procedure compare with existing robustness methods?
- **RQ2:** How does performance change as retrieval noise increases?
- **RQ3:** Does contrasting the strongest and weakest context rewrites improve synthesis?
- **RQ4:** How sensitive is performance to the number of synthesis iterations?
- **RQ5:** What does textual feedback contribute beyond fixed feedback?

5.1 Comparison with Robustness Training Baselines

Table **??** reports held-out EM and F1. Our method has the highest point estimate in every column, with the largest gains on DAE. The significance markers compare against LLaMA3.2-3B; they do not establish significance against every robustness baseline.

Table 2: Retrieval noise settings with progressively increasing distractor ratios.

Setting	Retrieved Context
0	Top1–5
1	Top1–4 + Rank50
2	Top1–3 + Rank49–50
3	Top1–2 + Rank48–50
4	Top1 + Rank47–50

Table 3: Held-out robustness comparison across five QA benchmarks. Best results in each column are shown in bold. * marks a significant improvement over LLaMA3.2-3B under a two-sided paired t-test on per-example scores ($p < 0.05$).

Model	NQ		POPQA		DAE		HotpotQA		2WikiMultiHopQA	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
LLaMA3.2-3B	0.441	0.573	0.667	0.765	0.052	0.157	0.434	0.555	0.414	0.501
LLaMA3-8B	0.426	0.566	0.642	0.772	0.169	0.315	0.402	0.549	0.290	0.438
Qwen3-4B	0.241	0.417	0.601	0.715	0.071	0.232	0.402	0.549	0.286	0.457
Qwen3-8B	0.333	0.496	0.587	0.732	0.097	0.205	0.483	0.612	0.470	0.563
RbFT	0.485	0.586	0.579	0.661	0.371	0.490	0.480	0.595	0.390	0.469
ATM	0.470	0.598	0.419	0.486	0.264	0.389	0.250	0.336	0.176	0.242
RAAT	0.382	0.483	0.704	0.747	0.161	0.275	0.394	0.500	0.405	0.480
Ours	**0.506***	**0.625***	**0.727***	**0.804**	**0.513***	**0.663***	**0.486**	**0.629***	**0.524**	**0.619***

Table 4: Effect of contrastive adversarial context selection. Comparison between the Best-vs-Worst (B-vs-W) and Best-vs-2ndBest (B-vs-2ndB) strategies. Δ denotes the relative performance change (%).

Dataset	Metric	B-vs-W	B-vs-2ndB	Δ (%)
Natural Questions	EM	0.498	0.492	-1.20
	F1	0.611	0.610	-0.16
PopQA	EM	0.711	0.703	-1.13
	F1	0.775	0.773	-0.26
DAE	EM	0.390	0.366	-6.15
	F1	0.501	0.478	-4.59
HotpotQA	EM	0.506	0.489	-3.36
	F1	0.642	0.625	-2.65
2WikiMultiHop QA	EM	0.471	0.471	0.00
	F1	0.566	0.571	+0.88

These results evaluate the complete synthesis-and-fine-tuning procedure. Because the methods follow different published training recipes, this comparison does not isolate the effect of synthesis under an equal training budget.

Table 5: Impact of offline synthesis iterations on DAE. Synthesis uses the fixed $3,000$-instance training set. Best Score is an internal synthesis indicator; EM and F1 measure held-out answer quality.

# Iterations	Best Score	EM	F1
1	0.478	0.367	0.482
2	0.505	0.380	0.501
3	0.538	0.390	0.501

5.2 Overall Robustness under Increasing Retrieval Noise

To address **RQ2**, we progressively replace high-ranked documents with low-ranked distractors, yielding Settings 0–4 in Table **??**.

Figure **??** reports the F1 performance across four benchmarks. F1 generally decreases as more high-ranked documents are replaced by distractors, illustrating sensitivity to retrieval quality. Vanilla RAG suffers the largest decline, especially on multi-hop datasets such as HotpotQA and 2WikiMultihopQA, while robustness-oriented methods partially alleviate this issue.

Our method remains competitive across the noise range on NQ, HotpotQA, and PopQA and is strongest under severe noise in Settings 3–4. RbFT performs better on 2WikiMultiHopQA, so the advantage is not universal. The overall pattern nevertheless indicates that training on target-aware difficult contexts reduces degradation under evidence dilution without changing the test-time pipeline.

5.3 Effect of Contrastive Adversarial Contexts

To address **RQ3**, we compare two contrastive pair construction strategies: *Best-vs-Worst*, which contrasts the strongest and weakest adversarial contexts, and *Best-vs-2ndBest*, which contrasts the two strongest adversarial contexts.

Best-vs-Worst is better on eight of ten dataset–metric combinations and tied on one; the only reversal is 2WikiMultiHopQA F1. The largest relative differences occur on DAE and HotpotQA. These results favor the wider contrast in most reported comparisons, but do not establish that it is uniformly optimal or directly measure feedback quality.

Table 6: Ablation study on different reward signaling strategies. We report EM and F1 scores across five QA benchmarks. Bold font indicates the best performance.

Strategy	Metric	NQ	PopQA	DAE	HotpotQA	2WikiMultiHopQA
w/o evidence	EM	0.489	0.709	0.497	0.448	0.489
	F1	0.620	0.787	0.630	0.602	0.608
w/o uncertainty	EM	0.495	0.718	0.494	0.440	0.486
	F1	0.611	0.792	0.651	0.615	0.574
w/o answer	EM	0.503	0.715	**0.513**	0.481	**0.524**
	F1	0.619	0.798	0.655	0.624	0.612
Full model	EM	**0.506**	**0.727**	**0.513**	**0.486**	**0.524**
	F1	**0.625**	**0.804**	**0.663**	**0.629**	**0.619**

Table 7: Effect of textual gradients. We compare the full method with a fixed-feedback variant; Δ is the relative change from fixed feedback to the full method (%).

Dataset	Metric	TextGrad	Fixed	Δ (%)
Natural Questions	EM	0.498	0.503	-0.99
	F1	0.611	0.596	+2.52
PopQA	EM	0.711	0.712	-0.14
	F1	0.775	0.784	-1.15
DAE	EM	0.390	0.363	+7.44
	F1	0.501	0.478	+4.81
HotpotQA	EM	0.506	0.486	+4.12
	F1	0.642	0.635	+1.10
2WikiMultiHop QA	EM	0.471	0.476	-1.05
	F1	0.566	0.563	+0.53

5.4 Impact of Synthesis Iterations

To address **RQ4**, we vary the number of offline textual-gradient iterations. As shown in Table **??**, EM consistently improves from 0.367 to 0.390 as the number of iterations increases, while most F1 gains are achieved after the second iteration ($0.482 \rightarrow 0.501$).

We additionally track the *Best Score*, an internal synthesis indicator. Because the cache retains previously evaluated candidates, its maximum score can increase simply by considering more candidates. This trend alone does not establish improved context quality or feedback quality; held-out EM and F1 provide the relevant downstream evidence. Overall, a small number of synthesis rounds captures most of the observed gain; these are offline text updates, not test-time model updates.

5.5 Role of Textual Feedback

To address **RQ5**, we replace pair-specific textual gradients with the same fixed instruction at every synthesis step. Table **??** shows the clearest benefit on DAE and HotpotQA, where TextGrad improves both EM and F1. Results are mixed on NQ, PopQA, and 2WikiMultiHopQA: the full method improves some metrics but slightly reduces others. Thus, the evidence supports textual feedback most strongly for the more difficult-context evaluations, not as an across-the-board improvement.

5.6 Ablation Study on Reward Signaling Strategies

Table **??** reports the ablation results of different reward components. The full reward is best or tied on every dataset–metric combination, but the component effects differ by dataset. Removing evidence preservation causes the largest F1 losses on DAE and HotpotQA, whereas removing the uncertainty term has the largest F1 effect on 2WikiMultiHopQA. Removing the answer term produces smaller or tied EM changes on some datasets but consistently lowers F1. These results support complementarity while avoiding the stronger, unsupported claim that every component improves every individual metric.

5.7 Deployment-Time Efficiency

The Qwen3-32B attacker is used only during offline synthesis. After LoRA fine-tuning, prediction uses the target LLaMA-3.2-3B RALM and its adapter weights through the usual retrieve-then-read pipeline. Table **??** reports wall-clock inference times on the same NQ test set, retrieval configuration, and hardware. Our method has the shortest observed time in this run. This is a measurement of the deployed model, not a claim of a faster inference algorithm: synthesis, scoring, and textual-feedback calls are absent from prediction. The table excludes offline synthesis and fine-tuning costs,

Table 8: Total NQ inference time under a shared retrieval and hardware configuration. The relative column is measured against our method.

Method	Time (s)	Relative
Ours	132.53	–
RbFT	146.22	+10.3%
ATM	150.65	+13.7%
LLaMA-3.2-3B-Instruct	198.14	+49.7%
RAAT	219.73	+65.8%

so it cannot establish lower total computational cost.

6 Conclusion

We introduced textual-gradient-guided adversarial data synthesis for robust retrieval-augmented generation. Frozen models iteratively construct and assess complete context rewrites, and the highest-reward candidates that pass an evidence check become supervision for LoRA fine-tuning. The resulting RALM requires no synthesis or adaptation during prediction. Experiments show improved held-out answer quality under the evaluated noisy and difficult-context settings, while ablations reveal that the effects of textual feedback vary across datasets. The findings motivate target-aware context construction as an offline training strategy; robustness to stronger malicious attacks remains untested.

Limitations

The synthesis stage requires labeled training instances to score answer failure and check evidence preservation; it is not directly applicable to unlabeled queries. The trained RALM, however, requires no reference answer at prediction time. Offline synthesis relies on Qwen3-32B and repeated model calls. We do not report the full synthesis cost or peak memory, and the inference times in Table **??** do not account for these resources. Trainable baselines follow their published recipes, so offline training compute is not equalized.

The evaluation covers short-form open-domain QA, difficult contexts, and naturally retrieved distractors. Results are conditioned on a common context-sufficiency filter and should not be interpreted as performance on unfiltered retrieval outputs. They do not establish robustness to corpus poisoning, malicious evidence injection, targeted retrieval attacks, long-form generation, multimodal RAG, or complete retrieval failure. Finally, passing an LLM-based evidence check is not a guarantee of factual preservation: judge errors can admit invalid rewrites or reject valid ones. Human validation and stronger entailment checks remain important for assessing this limitation.

A Experiment Details

We compare against both unadapted instruction-tuned backbones and methods designed specifically for robustness to retrieval noise. Every model is evaluated on the same retained examples and receives the same retrieved contexts.

Vanilla LLMs. We use the following models without additional robustness training:

- **LLaMA-3.2-3B-Instruct** and **LLaMA-3-8B**;
- **Qwen3-4B** and **Qwen3-8B**.

They measure how much robustness arises from the base model alone under the common RAG pipeline (**??**).

Robust RAG Methods. We reproduce three training-based defenses with LLaMA-3.2-3B-Instruct as the common target backbone:

- **RbFT** improves defect detection and utility extraction through robust fine-tuning (**?**);
- **ATM** uses multi-agent adversarial tuning (**?**); and
- **RAAT** trains on adaptively selected retrieval-noise patterns (**?**).

For each, we follow the training-set size and LoRA configuration specified by its original paper. We do not use evaluation examples to adapt any baseline.

A.1 Implementation Details

We use Qwen3-32B as the frozen attacker and LLaMA-3.2-3B-Instruct as the target RALM. We randomly sample 3,000 instances from the *training splits* of NQ and HotpotQA at a 2:1 ratio. For each instance, the retriever returns the top-5 passages; the same context-sufficiency criterion used in evaluation selects source contexts judged to support the reference answer. The initial pool size is $n = 4$, each round generates $N = 4$ new candidates, and the default number of synthesis iterations is 3. The reward weights are fixed to $\alpha = 1.0$, $\beta = 0.5$, and $\gamma = 2.0$ for every dataset.

We fine-tune the target with LLaMA-Factory (**?**). LoRA is applied to all linear layers with rank $r = 16$ and scaling factor $\alpha = 32$. Training uses `bfloat16`, AdamW, a cosine learning-rate schedule, an initial learning rate of 5×10^{-5}, and 3 epochs. The per-device batch size is 2 with 8 gradient-accumulation steps, and the maximum sequence length is 4096. FlashAttention 2 is enabled (**?**).

B Synthesis Procedure

For clarity, the complete offline procedure for one labeled training instance is:

1. Retrieve $\mathcal{R}_K(q)$ and serialize all K passages with q and y^*.

2. Sample n full-context rewrites from the frozen attacker for initial evaluation.

3. Evaluate each newly generated candidate d_i by obtaining a response y_i from the frozen target RALM conditioned on (q, d_i). Compute the answer, uncertainty, and evidence rewards and store the evaluated candidate and its score in the cache.

4. Select the highest- and lowest-reward cached contexts, generate pair-specific textual feedback, and sample N new full-context rewrites. Evaluate these candidates as in the preceding step before using them in further comparisons or final selection. Repeat the refinement according to the synthesis-iteration setting.

5. After synthesis, identify the cached candidates that pass the final evidence check. Only these candidates are eligible; select a highest-reward candidate d^* from this subset.

6. Collect the selected (q, d^*, y^*) triples and train the target RALM with supervised LoRA fine-tuning. Held-out answer generation then uses the trained target with no synthesis calls or parameter updates.

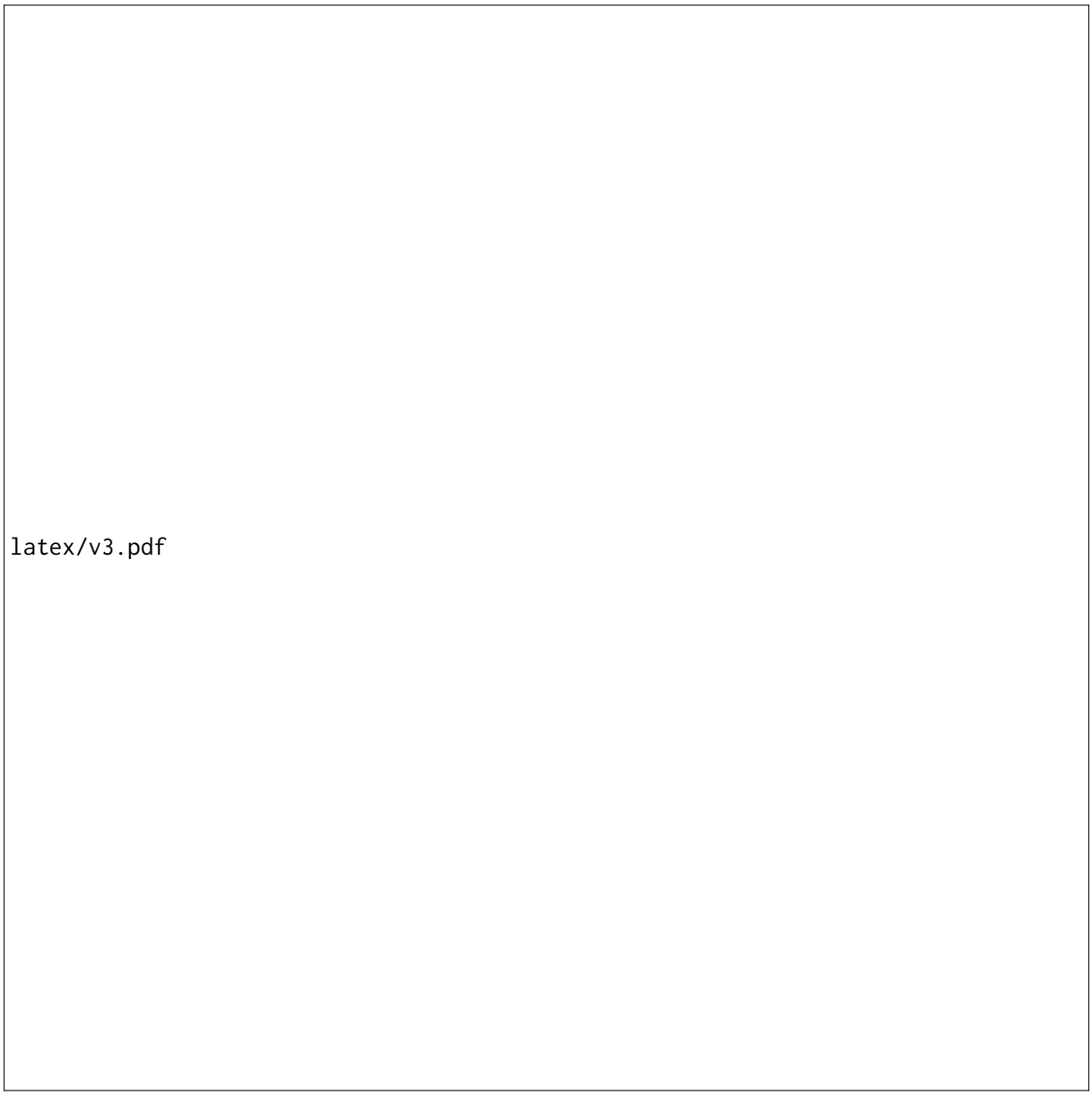


Figure 2: Overview of TAGS. TAGS first synthesizes query-specific adversarial contexts through an iterative generation–evaluation–feedback process. A frozen auxiliary attacker model generates candidate contexts, and textual gradients derived from contrastive feedback guide context optimization. The selected adversarial contexts are then used for LoRA-based robust fine-tuning of the target RALM. During inference, only the retriever and the fine-tuned RALM are required.

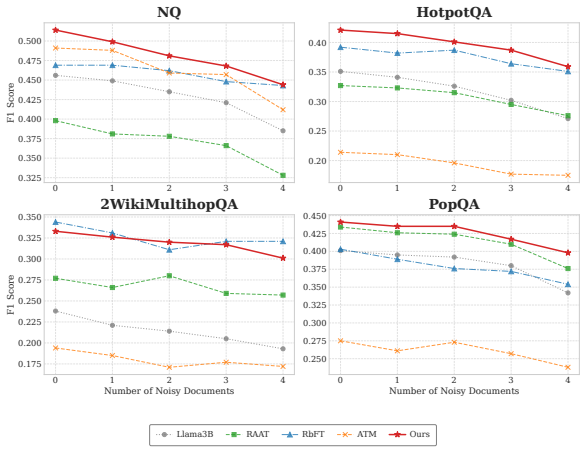


Figure 3: F1 as the number of low-ranked distractors increases on NQ, HotpotQA, 2WikiMultiHopQA, and PopQA. All methods receive the same retrieved contexts at each noise level.